АКОС
ИДЗ №2
Вариант 39. Регулируемый перекрёсток

Выполнил: студент БПИ252 Гришин Алексей Андреевич

Содержание

1 Условие и структура программы **2**

2 Интерфейс, конфигурация и время **3**

3 Синхронизация и управление движением **4**

4 Ресурсы, остановка и статистика **6**

5 Проверки и реальные результаты **7**

6 Требования и критическая оценка **8**

A Пользовательские промпты **9**

1 Условие и структура программы

Программа на C17 моделирует автомобили, пешеходов, направления и светофорный контроллер отдельными POSIX-потоками. Взаимодействие организовано одним мьютексом состояния и семафорами уведомлений. Реализована основная версия задания; дополнительная программа с другими синхропримитивами в состав работы не входит.

Исходный код, конфигурации и тесты находятся в проекте `se-caos-hw-2`. Инструкция пользователя приведена в `README.md`.

1.1 Предметная модель

На регулируемый перекрёсток прибывают автомобили и пешеходы. Для автомобиля задаются направление и манёвр: прямо, направо или налево. У пешехода используется манёвр перехода. Каждое направление имеет очередь ограниченной вместимости, интенсивность прибытия и длительность пересечения.

Фаза светофора разрешает множество попарно совместимых направлений. Симметричная матрица конфликтов запрещает небезопасные сочетания. Между фазами выдерживается защитный интервал. Контроллер работает по фиксированному циклу либо выбирает фазу по времени ожидания и длинам очередей. Экстренный автомобиль может запросить приоритет; пешеходный запрос возникает при принятии пешехода в очередь.

Обязательные события условия сохранены: прибытие, очередь или отказ, пешеходный запрос, сигналы, начало и конец фазы, переходный интервал, разрешение, въезд и выход. Для работы выбрано конфигурирование через текстовый файл и дублирование журнала в консоль и файл.

1.2 Распределение поведения по потокам

Поток	**Поведение**
Главный	Подготовка, запуск, приём сигналов, снимки состояния, сбор рабочих потоков и итоговая статистика.
Контроллер	Выбор фаз, закрытие въезда, ожидание освобождения зоны, выдерживание защитного интервала.
Направление	Независимая генерация прибытия, проверка очереди, создание участников и сбор завершившихся потоков своего направления.
Автомобиль или пешеход	Ожидание головы FIFO и допуска, занятие зоны, собственное ожидание срока проезда и регистрация выхода.

Очередь, фаза, сигнал и матрица конфликтов являются общими данными, а не отдельными активными сущностями. Их состояние меняют перечисленные потоки. В отличие от ИДЗ №1, главный цикл не выполняет проезды и переключения за участников. Совместимые направления действительно могут одновременно находиться в зоне.

1.3 Модули

Файл	Назначение
src/main.c	Аргументы, сигналы, открытие и закрытие журнала.
src/config.c, src/crossroad.h	Парсер, проверка конфигурации и предметные типы.
src/simulation.c	Запуск и сбор потоков, приём сигналов, журнал и итог.
src/simulation.h, src/simulation_state.c	Реестр потоков, примитивы, причина остановки и инварианты.
src/simulation_time.c	Монотонное время и ожидания вне мьютекса.
src/participants.c	Направления, генераторы и поведение участников.
src/controller.c	Стратегии и автомат контроллера.
src/io.c	Ввод-вывод через файловые дескрипторы.

2 Интерфейс, конфигурация и время

2.1 Сборка и варианты запуска

Нужны CMake 3.20+, компилятор C17 с POSIX Threads и Python 3.8+ для тестов. Сборка использует `Threads::Threads`, стандарт C17, `_POSIX_C_SOURCE=200809L` и предупреждения `-Wall -Wextra -Wpedantic -Wconversion -Wshadow`.

```
./tests/run.sh asan
./tests/run.sh tsan
./tests/run.sh none
./build/crossroad data/demo.conf results/demo.log
./build/crossroad --help
```

Режимы создают отдельные каталоги `build`, `build-tsan` и `build-plain`. ASan/UBSan и TSan одновременно не включаются. UBSan настроен завершать выполнение при ошибке. Скрипт запускает CTest, затем сохраняет журналы четырёх примеров. Переменные `CC`, `BUILD_DIR` и `RESULTS_DIR` позволяют изменить компилятор и пути. На Linux применяются те же команды, например `CC=gcc ./tests/run.sh asan`.

Интерфейс программы: `crossroad CONFIG [LOG]`. Без `LOG` используется `crossroad.log`. Для сборки без тестов достаточно `cmake -S . -B build-release -DBUILD_TESTING=OFF`, затем `cmake -build build-release -parallel`; Python не требуется.

2.2 Формат входных данных

```
stream NAME KIND DIRECTION MANEUVER CAPACITY CROSSING_TICKS RATE
conflict NAME_A NAME_B VALUE
phase NAME STREAM_1 STREAM_2 ...
```

`KIND` принимает значения `car`, `pedestrian`; направления — `north`, `east`, `south`, `west`. Манёвры автомобилей: `straight`, `right`, `left`; пешеходов: `walk`. `RATE` задаётся в тысячных долях на такт. Например, `1000` означает попытку прибытия на каждом обработанном такте.

Поддерживаются до 16 направлений и 16 фаз. Вместимость очереди — от 1 до 64; длительность пересечения — от 1 до 1000 тактов и не больше `max_green`. Участник в зоне не занимает место в очереди. По умолчанию все пары конфликтуют; `conflict N S 0` разрешает совместный проезд, единица запрещает его. Каждое направление должно обслуживаться фазой, содержащей только совместимые направления. Неверные значения отклоняются до запуска потоков.

Параметр	Смысл	По умолчанию
`ticks`, `target`	Период и целевое число выходов; ноль отключает соответствующий предел.	120; 0
`seed`	Начальное значение генераторов, от 0 до $2^{32}-1$.	39
`strategy`	`fixed` или `adaptive`.	`adaptive`
`min_green`, `max_green`	Нижняя и верхняя границы окна допуска, в тактах.	4; 10
`clearance`	Защитный интервал после освобождения зоны.	2
`priority`	Включение приоритета экстренных автомобилей.	1
`emergency_rate`	Доля экстренных автомобилей на 1000.	20
`tick_ms`, `delay_ms`	Реальная длительность такта и период снимков, в миллисекундах.	1; 0

`ticks` и `target` ограничены 10^9; длительности фаз — диапазоном $1\dots10000$ и условием `min_green <= max_green`. `tick_ms` находится в диапазоне $1\dots60000$, `delay_ms` — $0\dots60000$. Файл допускает комментарии после символа `##`; повторный параметр заменяет предыдущее значение. Входной файл нельзя использовать как журнал, в том числе через жёсткую ссылку.

2.3 Единица модельного времени

Все потоки используют общее начало отсчёта и `CLOCK_MONOTONIC`:

$$\Delta = \texttt{tick_ms} \cdot 10^6 \text{ нс}, \qquad t = \left\lfloor \frac{\tau - \tau_0}{\Delta} \right\rfloor .$$

Сроки хранятся в наносекундах. Если въезд зарегистрирован в момент a, выход разрешён не раньше $a + \texttt{crossingTicks} \cdot \Delta$. Участник сам ожидает этот срок вне мьютекса. Позднее пробуждение допустимо, ранний выход запрещён.

При `tick_ms 10` проезд в три такта занимает не менее 30 мс. `delay_ms` ограничивает только сообщения «Состояние». При нулевом значении снимок появляется не чаще одного раза за такт; предметные события не задерживаются. Ускоренный режим — `tick_ms 1`, `delay_ms 0`; для наблюдения удобно увеличить оба параметра до 100 мс.

На направление приходится не более одной попытки прибытия за текущий такт. Пропущенные из-за планировщика такты не догоняются пачкой. Состояние SplitMix64 у каждого направления собственное: `((uint64_t)seed « 32) | stream_index`. На такт резервируются два числа — прибытие и экстренность, независимо от заполнения очереди и результата первой проверки. Пропуск тактов продвигает состояние арифметически, не сдвигая решения будущих тактов. При одинаковом `seed` воспроизводятся возможные входные решения; фактически обработанные такты и общий порядок событий могут различаться.

3 Синхронизация и управление движением

3.1 Общее состояние и примитивы

Конфигурация после загрузки неизменяема. Очереди, сигналы, занятость зоны, статистика, реестр потоков и причина остановки защищены одним мьютексом. Функции с суффиксом `Locked` предполагают, что вызывающий уже захватил его. Проверка инвариантов выполняется после согласованного изменения состояния.

Каждый рабочий контекст имеет отдельный семафор уведомлений со значением ноль. Используются именованные POSIX-семафоры: имя удаляется сразу после открытия, объект живёт до `sem_close`. Такое решение работает и в Linux, и в macOS. Объекты семафоров не копируются вместе с данными `Participant`.

Сначала создаются ожидающие направления и контроллер. Главный поток задаёт начало времени и пробуждает контроллер. Тот включает первую фазу и выпускает направления. Новое прибытие уведомляет контроллер; смена фазы и выход будят головы очередей; завершившийся участник будит своё направление для сбора.

Без конечного срока поток блокируется в `sem_wait`. При ожидании срока `SimulationWait` проверяет семафор через `sem_trywait` и выполняет короткий `nanosleep` не более 1 мс. Срок проверяется заново по монотонным часам, поэтому уведомление и прерывание сна не сокращают длительность проезда. Ожидания и `pthread_join` не удерживают мьютекс.

3.2 Допуск как одна защищённая операция

После пробуждения участник проверяет весь предикат:

1. остановка и граница периода ещё не наступили;
2. состояние контроллера — `GREEN`, сигнал направления разрешён, срок `max_green` не истёк;
3. участник является головой FIFO, направление свободно, конфликтующих проездов нет;
4. при заданной цели $P + A < T$, где P — прошедшие, A — активные, T — цель.

Удаление головы очереди, занятие зоны и регистрация въезда выполняются до освобождения того же мьютекса. Контроллер закрывает въезд под ним же. Таким образом, участник либо успел занять зону до закрытия, либо остался в очереди. Устаревшее уведомление не даёт права въехать.

Внутри направления порядок всегда FIFO. Экстренный автомобиль может повлиять на выбор фазы даже из глубины очереди, но не обгоняет автомобили перед собой. Одновременно в одном направлении находится не более одного пересекающего участника; разные направления могут пересекать зону при совместимости.

3.3 Автомат контроллера



В `GREEN` действует окно допуска. До `min_green` запросы не прерывают фазу; затем смену может инициировать адаптация или экстренная заявка другой фазы. При достижении `max_green` новый въезд закрывается в любом случае. Участник дополнительно проверяет сам срок, даже если контроллер ещё не получил процессор.

В `DRAIN` контроллер ждёт фактического выхода последнего активного участника. Зелёные сигналы сохраняются для уже движущихся, но новые въезды запрещены. При освобождении зоны сигналы выключаются и начинается `CLEARANCE`. Следующая фаза включается не раньше конца полного интервала, даже если повторно выбрана та же фаза.

Например, при такте 100 мс, `max_green 4`, проезде в три такта и `clearance 2` участник может въехать в 300 мс. В 400 мс въезд закрывается, выход возможен не раньше 600 мс, следующая фаза — не раньше 800 мс. Задержка фактического выхода сдвигает начало защитного интервала.

Приоритет выбирает старейшую экстренную заявку по глобальному идентификатору. Без неё фиксированная стратегия обходит фазы циклически. Адаптивная сравнивает возраст голов ожидающих очередей, затем суммарную длину очередей, затем циклический порядок от следующей фиксированной фазы.

4 Ресурсы, остановка и статистика

4.1 Время жизни участника

При вместимости C_i направлению i выделено $C_i + 1$ неподвижных контекстов. Общее число одновременно несобранных участников ограничено

$$K = \sum_{i=1}^{n} (C_i + 1), \qquad N_{\text{threads}} \le 2 + n + K.$$

Вторая формула включает главный поток, контроллер и направления. В лимит входят завершившиеся потоки до успешного `pthread_join`. Число накопленных ресурсов не растёт с числом ранее прошедших участников.

Направление под мьютексом проверяет очередь и резервирует контекст. Затем вызывается `pthread_create`; при успехе регистрируются очередь и счётчики. При отказе идентификатор контекста сбрасывается, регистрируется отказ участнику и общая остановка с ошибкой выполнения. Частично подготовленные семафоры освобождаются при ошибке инициализации.

Перед возвратом участник устанавливает `finished`. Направление копирует идентификатор потока под мьютексом и собирает его вне критической секции. Затем очищаются запись и старые уведомления семафора; только после этого разрешено переиспользование. Свободной записи соответствует `participantId == 0`, что проверяется инвариантом. При остановке главный поток сначала собирает контроллер и направления, затем оставшихся участников, исключая двойной сбор.

4.2 Остановка и сигналы

Сохраняется первая установленная причина:

- период наблюдения истёк;
- зарегистрировано ровно `target` выходов;
- принят `SIGINT` или `SIGTERM`;
- при нулевых интенсивностях нет участников, а цель не достигнута;
- исчерпаны входы без цели либо обнаружена ошибка.

`SIGINT` и `SIGTERM` блокируются до инициализации модели. Рабочие потоки наследуют маску. Главный цикл проверяет `sigpending` и принимает ожидающий сигнал через `sigwait` вне мьютекса. После этого обычный код устанавливает защищённое состояние остановки и уведомляет все созданные потоки. Сигнал, ожидающий до запуска, принимается до создания рабочих потоков. Повторные сигналы остаются заблокированными до закрытия журнала и не прерывают сбор.

Порядок завершения: запретить новые прибытия и допуск, разбудить ожидающих, прервать ожидания сроков, собрать потоки, вывести согласованный итог, уничтожить примитивы и закрыть журнал. Непредвиденная ошибка владения мьютексом или `pthread_join` приводит к `abort`, поскольку продолжение могло бы освободить состояние работающего потока.

4.3 Граница периода и баланс

Для периода $B = \texttt{ticks} \cdot \Delta$ новые прибытия и въезды допустимы только при $\tau < B$. Выход учитывается при регистрации $\tau \le B$. Остановка по периоду фиксируется при $\tau > B$; точная граница ещё допускает выход, но не новый въезд.

При остановке статистика замораживается. Участники, чьи выходы не зарегистрированы, остаются в категории «в зоне», даже если расчётный срок проезда уже прошёл. Пробуждение и сбор потока не меняют число прошедших. Для целевого режима проверка $P + A < T$ резервирует место в цели при въезде, поэтому параллельные выходы не превышают T.

Для каждого направления и модели выполняется баланс:

$$\text{прибыло} = \text{отказов} + \text{прошло} + \text{ожидают} + \text{в зоне}.$$

Ожидание прошедшего участника равно целой части $(\tau_{\text{enter}} - \tau_{\text{arrive}})/\Delta$. Среднее и максимум рассчитываются только по прошедшим. Время пересечения в ожидание не входит.

Коды возврата: 0 — штатное завершение; 1 — ошибка выполнения или вывода; 2 — неверные аргументы или конфигурация; 3 — недостижимая цель; 128 + номер сигнала — прерывание.

5 Проверки и реальные результаты

5.1 Независимое восстановление журнала

Регистрация события и запись целой строки в оба канала выполняются под мьютексом модели. У строки есть префикс `[t=N]`, возрастающий номер «событие» и «время_нс» от начала наблюдения. Номер различает события с одинаковым временем. `WriteAll` повторяет короткие записи и прерванные `EINTR` вызовы. Ошибки консоли и файла учитываются отдельно.

`tests/check_log.py` восстанавливает очереди, сигналы и активные проезды. Он проверяет FIFO, конфликты, вместимость, порядок номеров, время проезда и защитных интервалов, генерацию входов, резервирование цели и всю статистику. Сравниваются консоль и файл одного запуска. Побайтовое совпадение разных запусков и точные такты выхода не являются требованиями к асинхронной модели.

Тест CTest	**Проверяемые свойства**
`state`	Первая причина остановки, пробуждение, инварианты, уничтожение после сборов; граница периода до наносекунды с подставными часами.
`participants`	FIFO, устаревшие уведомления, задержки, приоритет, ожидание `DRAIN`, независимость генераторов от чередования обращений.
`lifecycle`	Два запуска по 1000 проездов при пяти контекстах; сбор и переиспользование, отказы создания потоков и семафоров, ожидающие сигналы.
`scenarios`	17 сценарных тестов: входные данные, стратегии, нагрузка, цель, переполнение, сигналы в трёх состояниях и ошибки вывода.
`stress`	Повторные запуски обеих стратегий с разными `seed`, проверкой журнала и тайм-аутом.

5.2 Платформы и нагрузка

Проверки выполнены 23 октября 2026 года. На macOS arm64 использован Homebrew Clang 22.1.8. В Linux — Debian 12 arm64, GCC 12.2.0, glibc 2.36, ядро LinuxKit 6.12.67 в локальном контейнере. Docker использовался как среда проверки и не является зависимостью проекта.

Платформа	**Режим**	**CTest**	**Нагрузка**
macOS arm64	ASan/UBSan	5/5	48/48
macOS arm64	TSan	5/5	48/48
Debian 12 arm64	Без санитайзеров	5/5	48/48
Debian 12 arm64	ASan/UBSan	5/5	48/48

Каждый набор нагрузки содержит две стратегии, два сценария, четыре значения `seed` (0, 1, 39, 4294967295) и три повтора. Первый сценарий заканчивается по периоду при заполненных

очередях и содержит пешеходов, второй — после 80 выходов. Всего выполнено $2 \cdot 2 \cdot 4 \cdot 3 \cdot 4 = 192$ нагрузочных запуска. Ошибок санитайзеров и нарушений проверяемых инвариантов не обнаружено.

Один запуск нагрузки ограничен 10 секундами, весь тест — 180 секундами. Итог сохраняется в `<build>/stress/summary.json`; при ошибке конфигурация, журнал и диагностические файлы остаются в `<build>/stress/failure/`. Отдельно проверены срабатывание тайм-аута, сохранение этих файлов и отказ CMake при совместном включении санитайзеров. В Linux выполнен тест `/dev/full`; на macOS он пропускается.

5.3 Примеры фактических итогов

Ниже — сохранённые журналы четырёх конфигураций после запуска с TSan на macOS, `tick_ms=1`, `delay_ms=0`. Таблица описывает конкретные измерения, а не эталон для побайтового сравнения.

Сценарий	Прибыло	Прошло	Отказы	Ждут	В зоне	Среднее	Макс.
`demo`	66	33	7	26	0	23,70	55
`hand`	16	2	10	3	1	0,50	1
`emergency`	40	6	2	31	1	7,50	15
`compatible`	12	4	0	6	2	0,50	1

В `compatible` два совместимых направления остались в зоне на границе периода. Они не были засчитаны как прошедшие при последующем сборе потоков. Для `hand` баланс равен $16 = 10 + 2 + 3 + 1$. Журналы дополнительно обработаны независимым проверяющим скриптом.

6 Требования и критическая оценка

6.1 Соответствие требованиям

Пункт требований	Реализация
Основные 1, 8	ФИО, группа, вариант, условие, интерфейс, сценарий потоков и фактические результаты приведены в отчёте.
Основные 2–5	C17, libc, pthread, POSIX-ввод-вывод, наблюдаемые события и все предусмотренные условия завершения.
Основные 6, 7	Контроллер, направления, автомобили и пешеходы имеют потоки; взаимодействие асинхронное через мьютекс и семафоры.
Дополнительные 1–5	Поясняющие комментарии, дескрипторы файлов, сигналы, сохранение журнала и тестовые наборы со скриптами запуска.
Дополнительные 6, 7	Использование ИИ, фактические запросы и критический разбор изменений приведены ниже и в приложении А.
Дополнительный 8	Вторая программа с альтернативными синхропримитивами не реализована.

6.2 Использование ИИ и проверенные изменения

ИИ-ассистент (LLM) использовался при анализе условий, составлении плана, реализации потоков и синхронизации, подготовке проверяющего скрипта, сборочных сценариев и документации. Первичный код, предложенный нейросетью, содержал избыточные абстракции. Я провел глубокий рефакторинг, в результате которого оставил только необходимую и надежную логику.

A Пользовательские промпты

Промпт 1

Рассмотри допуск автомобиля как конкурентный протокол. Построй минимальное чередование действий двух потоков, при котором проверка зелёного сигнала вне критической секции приводит к въезду после закрытия фазы. Отметь точку линеаризации въезда и объясни, какие именно действия должны защищаться одним мьютексом. Затем предложи тест, который различит ошибочную и исправленную версии.

Промпт 2

Раздели свойства модели на безопасность и прогресс. Для FIFO, отсутствия конфликтов и завершения DRAIN укажи предпосылки, сохраняемый инвариант и возможный контрпример. Не считай справедливость планировщика гарантией pthread. Какие выводы подтверждает журнал, а какие нельзя получить из конечного числа успешных запусков?

Промпт 3

У нас отдельный SplitMix64 на направление и два числа на модельный такт. Предложи метаморфные тесты: перестановка обращений к направлениям, пропуск тактов, изменение вместимости очереди и доли экстренных автомобилей. Для каждого изменения сформулируй, какая часть входной последовательности обязана сохраниться, а какой наблюдаемый результат вправе измениться.

Промпт 4

Построй матрицу отказов: первая и последующая инициализация семафора, создание направления, контроллера и участника после нескольких успешных созданий. Для каждой точки перечисли уже принадлежащие программе ресурсы, порядок отката, баланс статистики и признаки безопасного повторного использования контекста. Предложи внедрение отказов без изменения рабочего API.

Промпт 5

Представь, что проверяющему доступен только текстовый журнал. Найди два различных исполнения, которые выглядят одинаково при целых тактах, но одно нарушает длительность проезда или защитного интервала. Определи минимальные добавочные поля, позволяющие обнаружить нарушение. Проверь, достаточно ли номера события и относительного монотонного времени.

Промпт 6

Опиши опыт с подставными монотонными часами для моментов B-1, B и B+1 нс. В зоне находится автомобиль с истёкшим расчётным сроком, но незарегистрированным выходом; в очереди ждёт следующий. Предскажи допустимые операции, итоговые категории и первую причину остановки. Отдельно рассмотри совпадение достижения цели с границей периода и поступление сигнала во время сбора потоков.